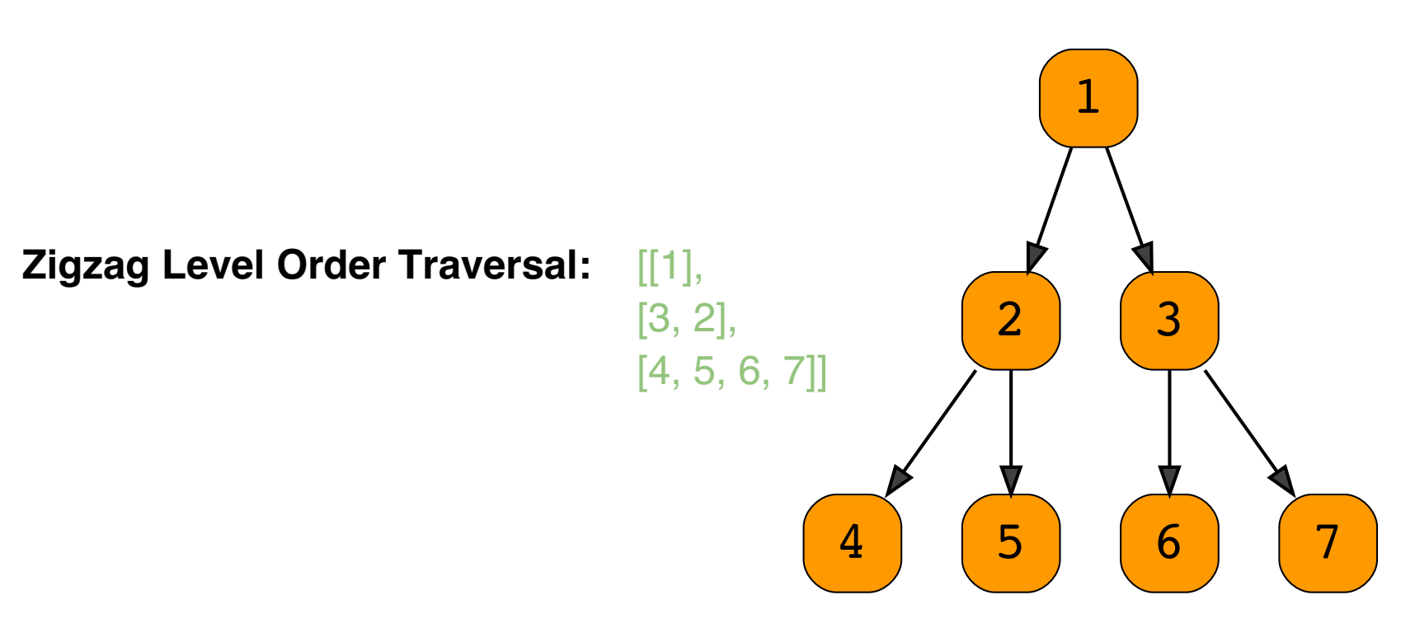
Solution: Zigzag Traversal

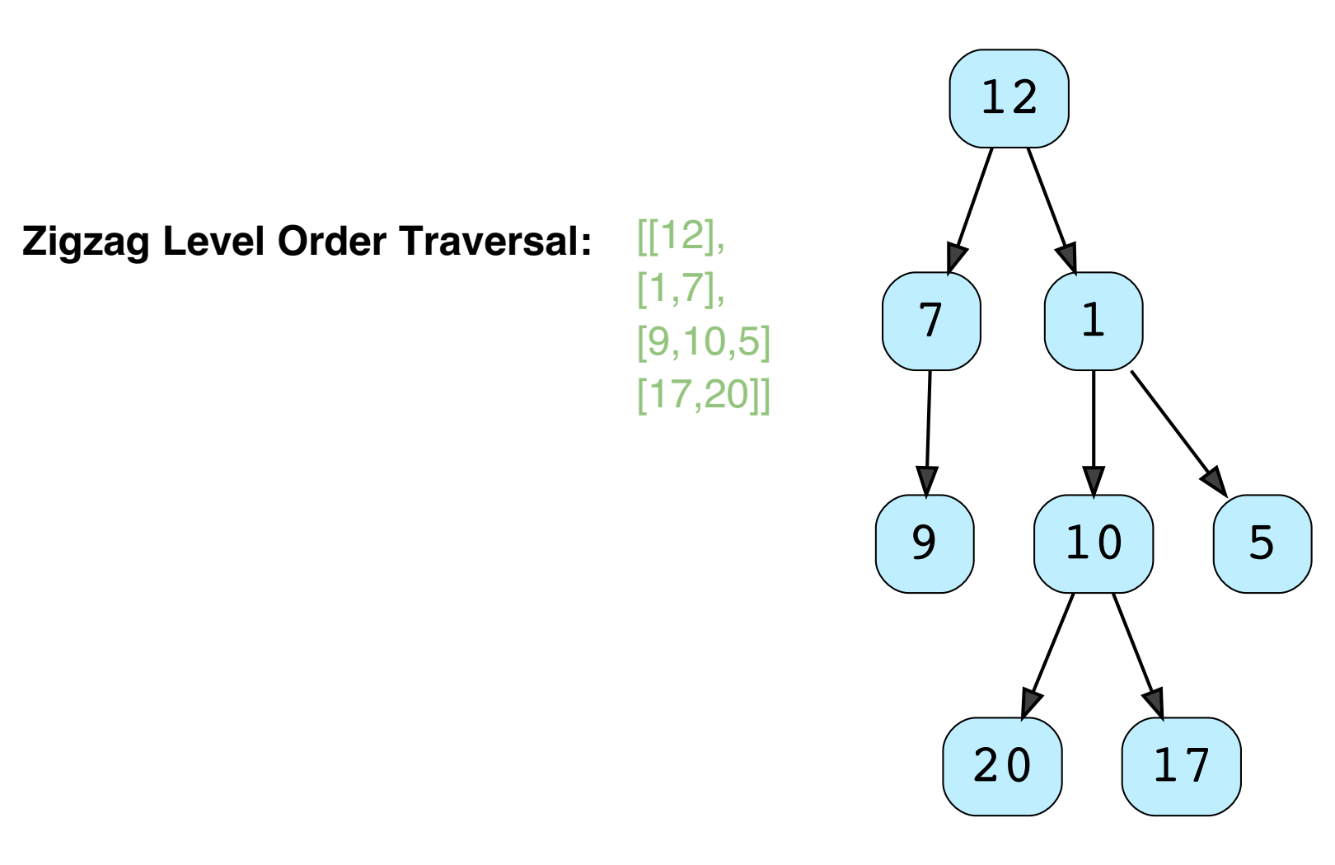
Problem Statement

Given a binary tree, populate an array to represent its zigzag level order traversal. You should populate the values of all **nodes of the first level from left to right**, then **right to left for the next level** and keep alternating in the same manner for the following levels.

Example 1:



Example 2:



Solution

This problem follows the **Binary Tree Level Order Traversal** pattern. We can follow the same **BFS** approach. The only additional step we have to keep in mind is to alternate the level order traversal, which means that for every other level, we will traverse similar to **Reverse Level Order Traversal.**

Here is the visual representation of the algorithm:

Zigzag Traversal

Code

Here is the code for this algorithm:

Java

```java
import java.util.*;

// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;

//   TreeNode(int x) {
//     val = x;
//   }
// };

class Solution {
  public List<List<Integer>> traverse(TreeNode root) {
    List<List<Integer>> result = new ArrayList<List<Integer>>();
    if (root == null)
      return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    boolean leftToRight = true;
```

Run Code Result Stdout

You must run your code first

Execute

Time Complexity

The time complexity of the above algorithm is O(N), where ‘N’ is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space Complexity

The space complexity of the above algorithm will be O(N) as we need to return a list containing the level order traversal. We will also need O(N) space for the queue. Since we can have a maximum of N/2 nodes at any level (this could happen only at the lowest level), therefore we will need O(N) space to store them in the queue.

← Previous
Zigzag Traversal (Medium)
Next →
Level Averages In A Binary Tree (Easy)
☐ Mark as Completed